Lab U2 - turtlesim built from source (`ros_tutorials`)

Aim. Compile ROS's own teaching simulator **turtlesim** from its source repository (C++, Qt5) and drive it through every interface type: topic, services, an action, and the `mimic` node that links two simulators.

Time. 45 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools, C++ workload). The repository is downloaded by `pixi run build-turtlesim`. Source: `upstream\ros_tutorials-jazzy` (branch `jazzy`), built into `upstream_ws\turtlesim`; the pixi package of the same name is shadowed by it.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and Visual Studio Build Tools (C++ workload); internet access once, to download `ros/ros_tutorials` from GitHub.

In this folder	
`upstream\`	the original repository (downloaded by the first build task, pinned commit); `upstream_ws\` its build
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
```

Run one lab at a time, and everything from this folder.

Theory

turtlesim draws one turtle on an 11 x 11 canvas.

Interface	Name	Type
topic in	`/turtle1/cmd_vel`	`geometry_msgs/Twist` (linear.x forward, angular.z turn)
topic out	`/turtle1/pose`	`turtlesim/Pose` (x, y, theta, velocities)
services	`/spawn`, `/kill`, `/clear`, `/reset`, `/turtle1/teleport_absolute`, `/turtle1/teleport_relative`, `/turtle1/set_pen`	`turtlesim/srv/...`
action	`/turtle1/rotate_absolute`	`turtlesim/action/RotateAbsolute` (goal `theta`; feedback `remaining`; result `delta`)

Two running simulators must differ in their **namespace**, otherwise their topics collide.

Procedure

Shortcut to run `ros2` with the source-built overlay (re-type it in every new terminal):

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
```

1. Build turtlesim from source (about 40 s; the first time it compiles C++ and Qt).

```
pixi run build-turtlesim
```

2. T1 - start it (a blue window with one turtle in the middle).

```
pixi run demo-turtlesim
```

3. T2 - prove it is *your* build, then read the pose. The first path must point into `upstream_ws\turtlesim`, the second (without the overlay) into the pixi environment (a folder under `C:\rlabs`).

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 pkg prefix turtlesim
pixi run ros pkg prefix turtlesim
ts ros2 topic echo /turtle1/pose --no-daemon --spin-time 5 --once
```

4. T2 - topic: move the turtle. One `Twist` message: 2 units/s forward, 1.8 rad/s turn, for one second (turtlesim stops the turtle after 1 s without commands).

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.8}}"
```

5. T2 - services. A second turtle, a jump, a red thick pen, a clean canvas, and removal of the second turtle. (`'off': 0` is quoted because YAML would read a bare `off` as a boolean.)

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 service call /spawn turtlesim/srv/Spawn "{x: 2.0, y: 2.0, theta: 0.0, name: turtle2}"
ts ros2 service call /turtle1/teleport_absolute turtlesim/srv/TeleportAbsolute "{x: 8.0, y: 8.0,
theta: 0.0}"
ts ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r: 255, g: 0, b: 0, width: 4, 'off':
0}"
ts ros2 service call /clear std_srvs/srv/Empty
ts ros2 service call /kill turtlesim/srv/Kill "{name: turtle2}"
```

6. T2 - action. Turn to 1.57 rad (90 degrees) and wait for the result.

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}"
```

7. T2 - a node of the package: `draw_square`. The turtle drives a square (about 20 s). Stop with Ctrl+C.

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim draw_square
```

```
# Ctrl+C in T1 and T2
```

8. Two simulators and `mimic`. T1: the normal turtlesim. T2: a second one in namespace `/turtlesim2`. T3: `mimic` copies the pose of `turtle1` of the first to the command input of the second.

```
pixi run demo-turtlesim
```

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim turtlesim_node --ros-args -r __ns:=/turtlesim2
```

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 run turtlesim mimic --ros-args -r input/pose:=/turtle1/pose -r
output/cmd_vel:=/turtlesim2/turtle1/cmd_vel
```

Now move the first turtle (T4); the second one follows it exactly:

```
function ts { pixi run cmd /c scripts\in_upstream.bat turtlesim @args }
ts ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 2.0}, angular: {z:
1.8}}"
Start-Sleep 3
ts ros2 topic echo /turtlesim2/turtle1/pose --no-daemon --spin-time 5 --once
```

```
# Ctrl+C in T1, T2, T3
```

Observations (recorded)

Item	Value
Start pose	(5.544, 5.544), theta 0
After `pub --once` (2.0, 1.8)	(6.60, 6.94), theta 1.81
`spawn`	reply `name='turtle2'` (a second `spawn` with the same name is refused - checked by `test-turtlesim`)
`rotate_absolute 1.57`	about 100 feedback messages (`remaining` from 1.57 to 0), result `delta` about -1.55, status SUCCEEDED
`mimic`	the pose of `/turtlesim2/turtle1` equals that of `/turtle1`

Verification

```
pixi run test-turtlesim
```

15 checks on the source-built package: the overlay shadows the pixi one; `cmd_vel` of 2.0 for one second moves 2.02 units; both teleport services; `spawn` (and refusal of a duplicate) and `kill`; `set_pen`; the `rotate_absolute` action; and `mimic` across two instances.

Questions

1. Why does the turtle move only for one second after a single `topic pub --once`?
2. Which interface type would you use to (a) ask for the position once, (b) rotate to an angle and be told when done, (c) stream velocities? What does turtlesim use?
3. Why must the second simulator run in a namespace? What would `mimic`'s two remaps look like if both simulators used namespaces?
4. How does `ros2 pkg prefix` prove which build you run?
5. *(Extension)* Write a node that draws a circle of radius 2 using only `/turtle1/cmd_vel`.

If it does not work

Symptom	Fix
`upstream_ws\turtlesim is not built`	`pixi run build-turtlesim`
Window is black / not drawn	wait for start-up; close other turtlesim windows from earlier labs
`Failed to populate field` in a service call	put quotes around the whole `{...}` and quote the key `'off'`